

```
django-bootstrap5==21.3
platformshconfig==2.4.0
soupsieve==2.3.2.post1
sqlparse==0.4.2
```

Learning Log already depends on specific versions of seven different packages, so it requires a matching environment to run properly on a remote server. (We installed three of these packages manually, and four of them were installed automatically as dependencies of these packages.)

When we deploy Learning Log, Platform.sh will install all the packages listed in *requirements.txt*, creating an environment with the same packages we're using locally. Because of this, we can be confident the deployed project will function just like it has on our local system. This approach to managing a project is critical as you start to build and maintain multiple projects on your system.

NOTE

If the version number for a package listed on your system differs from what's shown here, keep the version you have on your system.

Additional Deployment Requirements

The live server requires two additional packages. These packages are used to serve the project in a production environment, where many users can be making requests at the same time.

In the same directory where *requirements.txt* is saved, make a new file called *requirements_remote.txt*. Add the following two packages to it:

```
requirements # Requirements for live project.
_remote.txt  unicorn
             psycopg2
```

The unicorn package responds to requests as they come in to the remote server; this takes the place of the development server we've been using locally. The psycopg2 package is required to let Django manage the Postgres database that Platform.sh uses. *Postgres* is an open source database that's extremely well suited to production apps.

Adding Configuration Files

Every hosting platform requires some configuration for a project to run correctly on its servers. In this section, we'll add three configuration files:

.platform.app.yaml This is the main configuration file for the project. This tells Platform.sh what kind of project we're trying to deploy and what kinds of resources our project needs, and it includes commands for building the project on the server.

.platform/routes.yaml This file defines the routes to our project. When a request is received by Platform.sh, this is the configuration that helps direct these requests to our specific project.

.platform/services.yaml This file defines any additional services our project needs.